

# LA MÉMOIRE CACHE

M2 Secil : KINN9HU  
Pascal SAINRAT

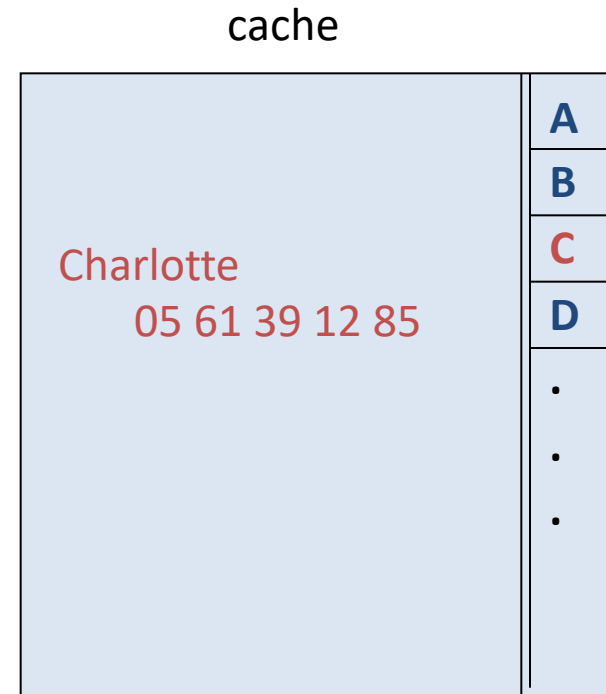


# But

- Accéder plus rapidement à certaines informations
  - celles auxquelles on accède le plus souvent
  - celles auxquelles on est susceptible d'accéder prochainement
- Exemples :
  - cache web
  - proxy
  - cache disque
  - **cache mémoire**

# Analogie (cache à accès direct)

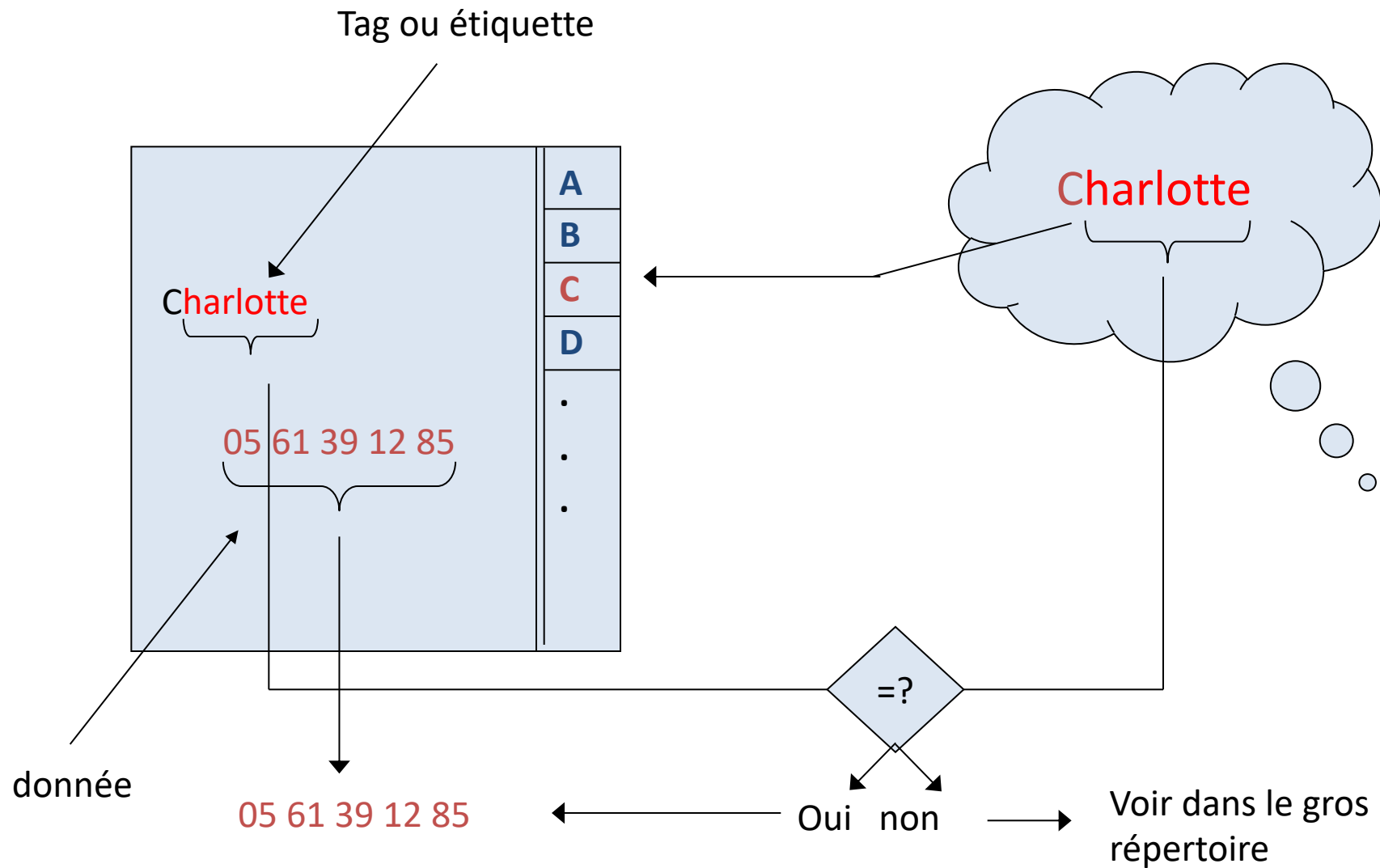
- Gros répertoire téléphonique
- Appel fréquent de quelques personnes seulement
- Cache du répertoire
  - 26 entrées, accès direct par première lettre
  - entrée C mémorise la dernière personne appelée dont le nom commence par C



répertoire

|                         |
|-------------------------|
|                         |
| Charlotte<br>0561391285 |
| Denis<br>0561392742     |
|                         |
| Christian<br>0422133345 |
|                         |

# Echec ou succès ?



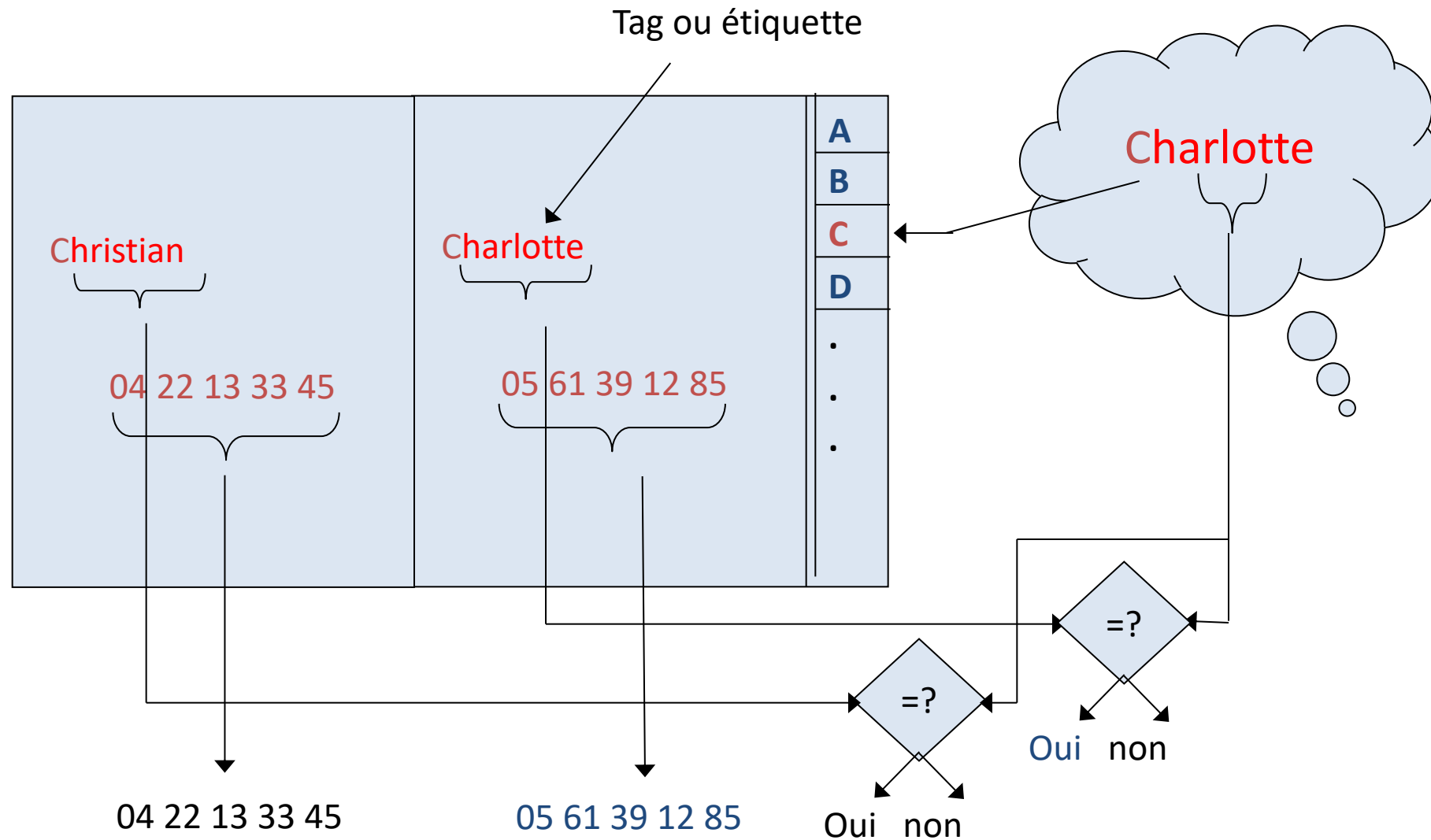
# Echec ou succès ?

- Problème : Charlotte et Christian

- Appel Charlotte
- Appel Christian → échec car dernier appel = Charlotte
- Appel Charlotte → échec car dernier appel = Christian
- Appel Christian → échec car dernier appel = Charlotte
- Appel Charlotte → échec car dernier appel = Christian
- Appel Christian → échec car dernier appel = Charlotte
- Appel Charlotte → échec car dernier appel = Christian
- Appel Christian → échec car dernier appel = Charlotte
- Appel Charlotte → échec car dernier appel = Christian
- Appel Charlotte → succès car dernier appel = Charlotte

Toutes les personnes dont le nom commence par C vont dans la même fiche.

# Associativité par ensembles



# Augmenter la probabilité de présence dans le cache

- Je dois appeler fréquemment Christian, Charlotte et Claire
  - Augmenter l'associativité (la taille d'un ensemble)
    - Comparaison de 2 prénoms → comparaison de 4 prénoms

|  |                |                |                |   |
|--|----------------|----------------|----------------|---|
|  | Claire         | Christian      | Charlotte      | A |
|  |                |                |                | B |
|  |                |                |                | C |
|  |                |                |                | D |
|  |                |                |                | . |
|  | 02 27 36 44 45 | 04 22 13 33 45 | 05 61 39 12 85 | . |
|  |                |                |                | . |
|  |                |                |                | . |

Risque de nombreux emplacements vides dans le cache ?

# Associativité totale

- Au maximum : un seul ensemble qui peut contenir n'importe quel prénom
  - plus difficile de trouver un prénom
    - pas d'index
    - besoin de comparer les 26 prénoms
  - Où mettre un nouveau prénom ?
    - emplacement vide
    - et s'ils sont tous pleins ?

|                             |   |
|-----------------------------|---|
| Charlotte<br>05 61 39 12 85 | 1 |
|                             | 2 |
|                             | 3 |
|                             | 4 |
|                             | . |
|                             | . |
|                             | . |

# En résumé

- Cache = copie d'une partie du gros répertoire
- Cache à accès direct
  - Très rapide
    - Je regarde la page C. Si ce n'est pas le bon, je regarde dans le gros répertoire.
    - Si j'appelle un prénom commençant par C, je le mets dans la page C (et écrase celui qui y était déjà).
- Cache associatif
  - Plusieurs emplacements pour chaque lettre
  - Moins rapide
    - besoin de regarder plusieurs entrées dans un ensemble
    - Si l'ensemble est plein, quelle entrée on remplace ?

# Localité temporelle

- Si, chaque fois que je téléphone à Christian, je téléphone à Denis
  - mettre Christian ET Denis ensemble dans le cache quand appel Christian et inversement (**localité temporelle**)
- Si, dans le gros répertoire, Ludovic et Marie sont l'un à côté de l'autre parce que quand j'appelle Ludovic, j'appelle aussi Marie
  - idem mais **localité spatiale** (dans le gros répertoire)

# Back to computer science

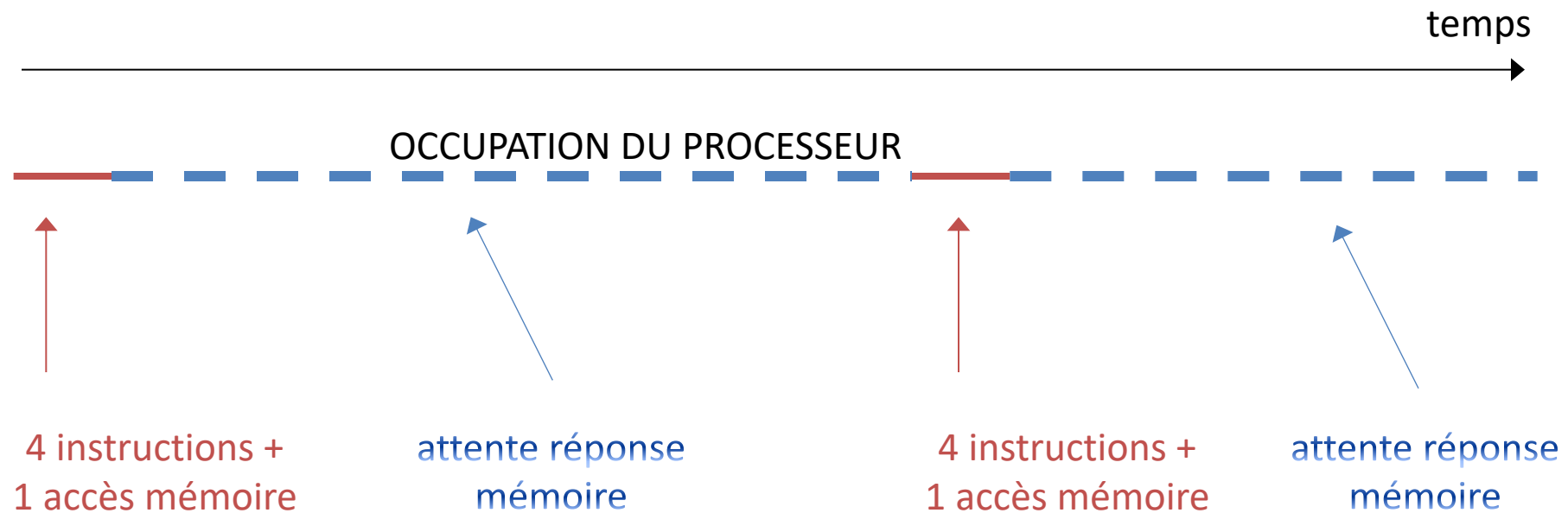
- Pourquoi a-t-on besoin de caches entre la mémoire et le processeur ?
- Comment ça marche ?

# Caractéristiques des mémoires

|                       | Temps d'accès | Nombre d'entrées | Coût par méga_octet |
|-----------------------|---------------|------------------|---------------------|
| Disque                | 10 ms         | 100 G            | 0,1 €               |
| Mémoire RAM dynamique | 150 ns        | 10 M             | 10                  |
| Mémoire RAM statique  | 10 ns         | 50 K             | 100                 |
| Registres             | 1 ns          | 64               | +++                 |

# Caractéristiques des programmes

- 20% de branchements
- 20% d'accès à la mémoire
- Temps d'accès à la mémoire centrale > 100 instructions
  - 100 instructions = 20 accès à la mémoire !



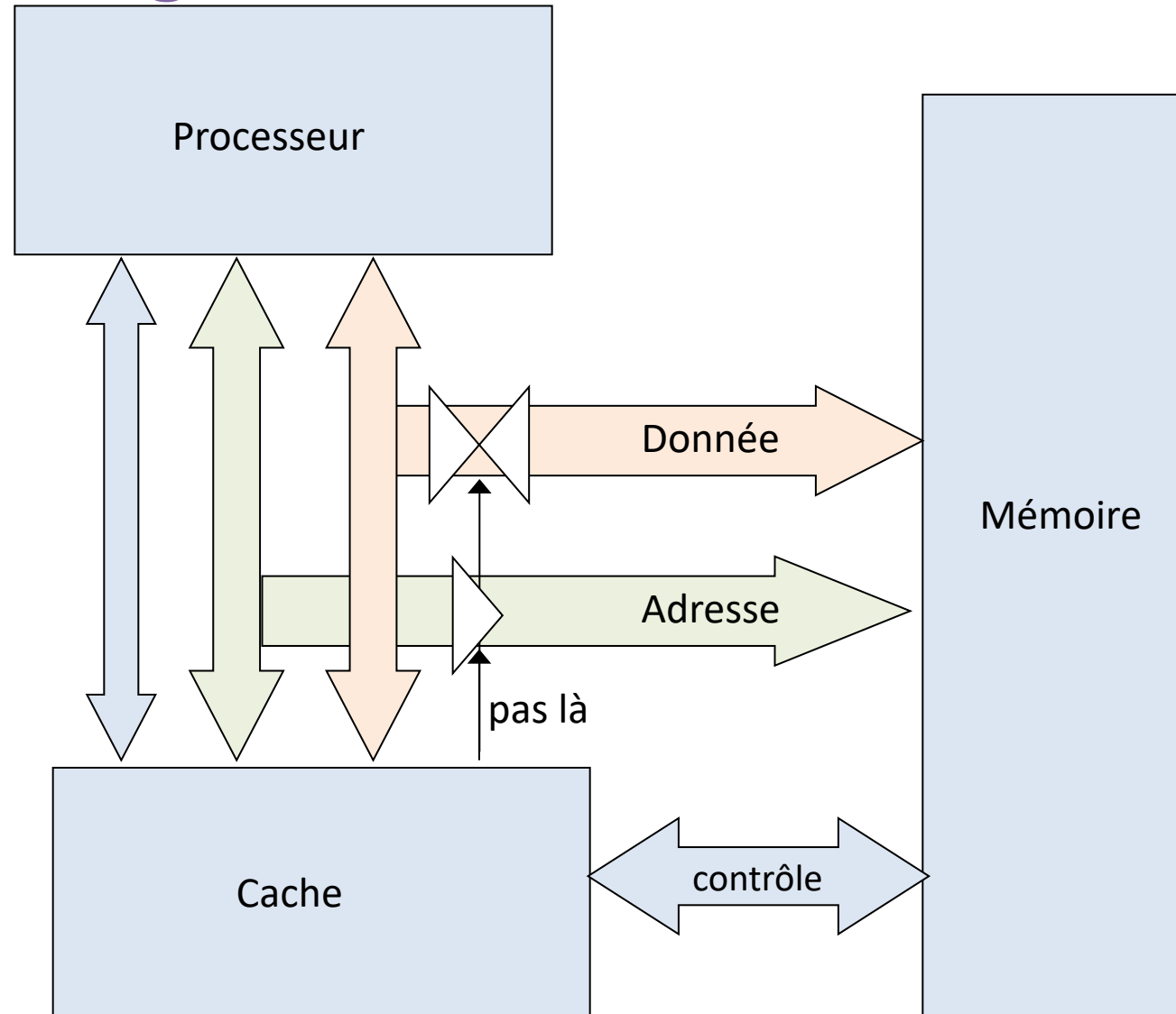
Et les instructions aussi doivent être chargées depuis la mémoire !

# Evolution

- La mémoire est de – en – rapide par rapport au processeur

| Année | Nombre d'instructions<br>pouvant être exécutées<br>durant un accès à la mémoire |
|-------|---------------------------------------------------------------------------------|
| 1995  | 100                                                                             |
| 2000  | 300                                                                             |
| 2005  | 600                                                                             |

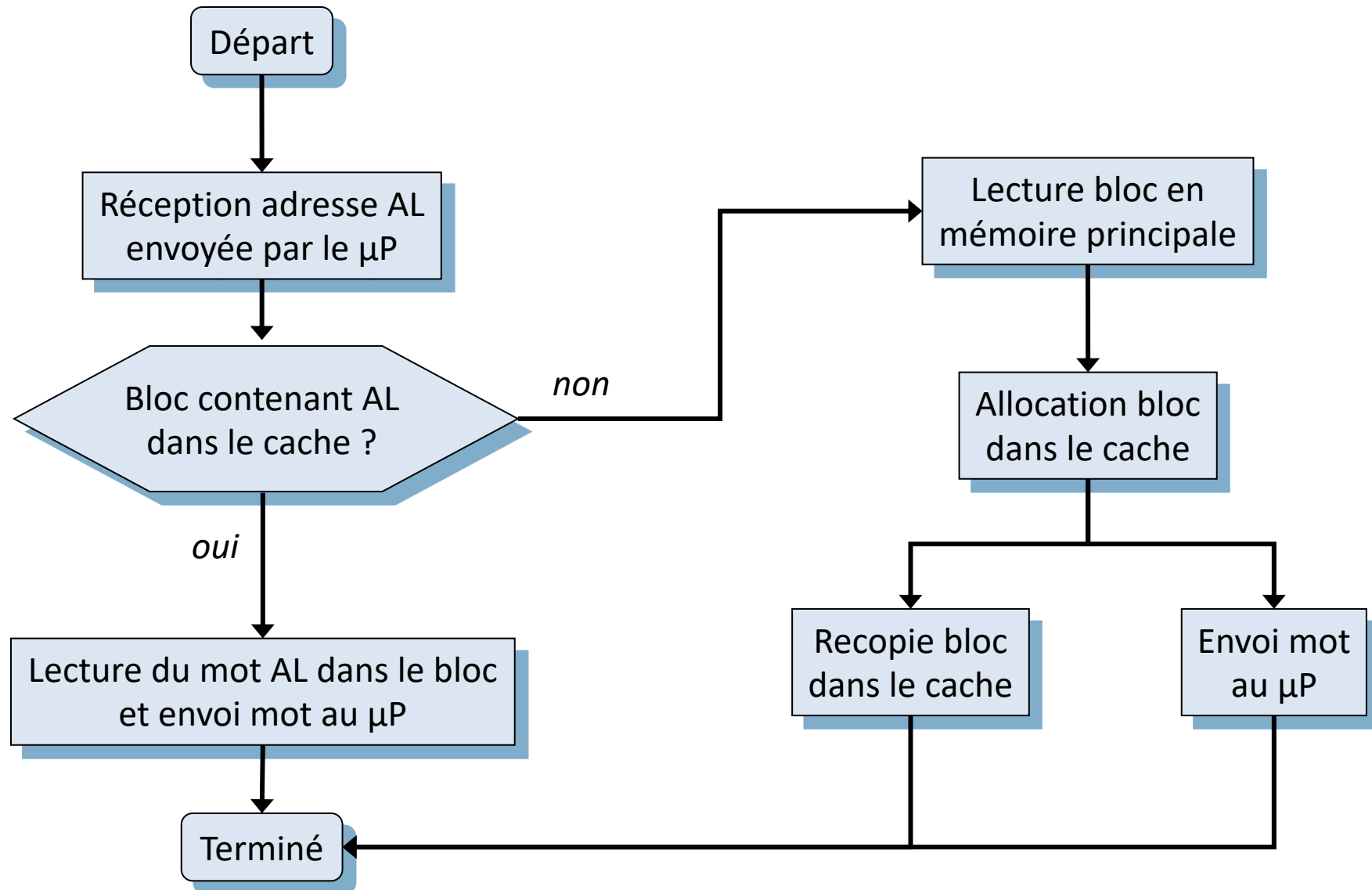
# Architecture générale



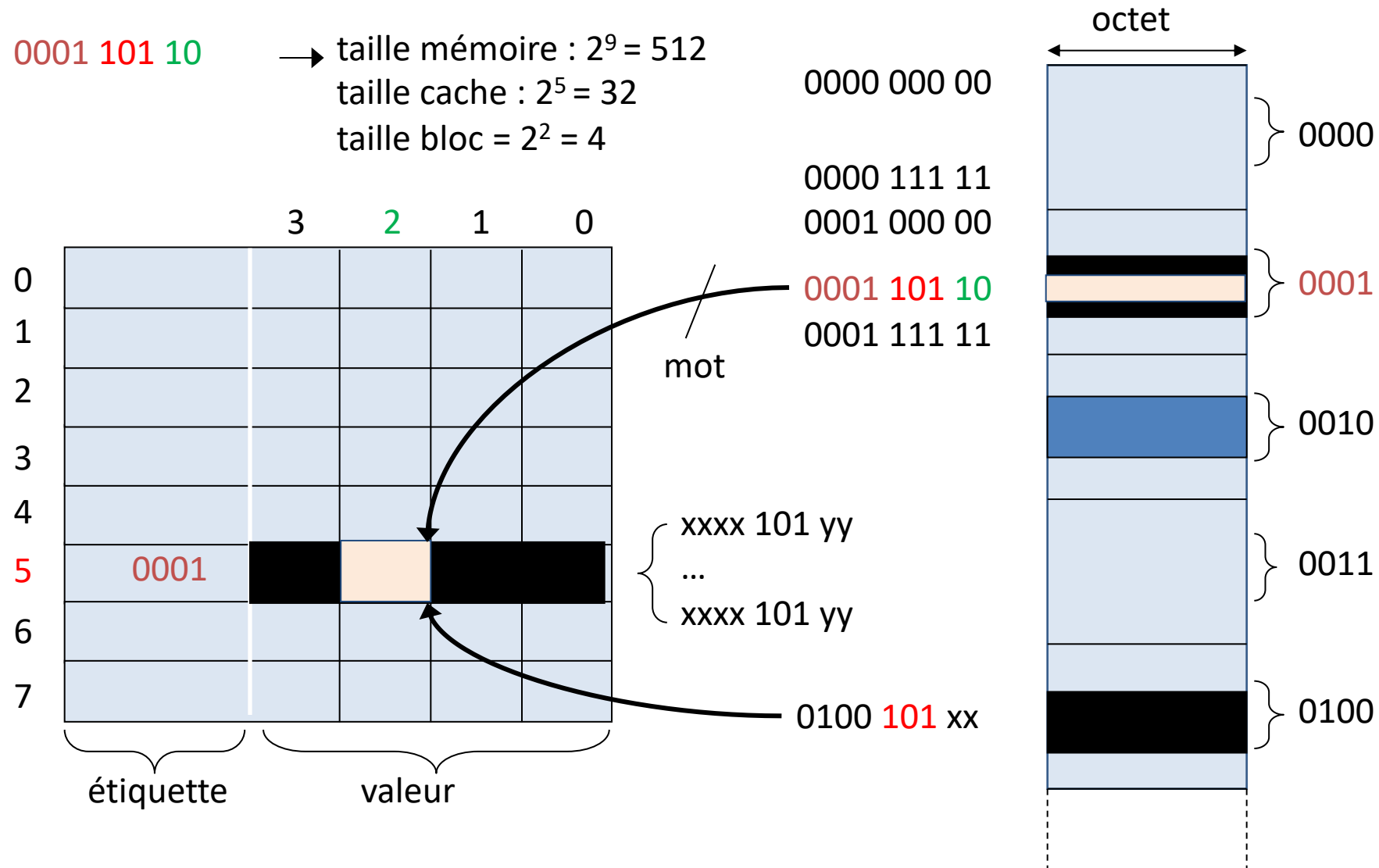
# Fonctionnement général

- Lecture par le processeur d'une instruction (ou une donnée)
- Recherche dans le cache (**comment ?**)
- Si succès, le cache renvoie l'instruction sur le bus de données
- Si échec, accès à la mémoire
  - la mémoire renvoie l'instruction sur le bus de données
  - l'instruction est automatiquement copiée dans le cache (**où ?**)
  - pour profiter de la localité spatiale
    - la mémoire renvoie l'instruction et ses voisines (**combien ? lesquelles ?**)
      - bloc
    - elles sont copiées dans le cache (**où ?**)
- Ecriture
  - **dans le cache ? dans la mémoire ? dans les 2 ?**

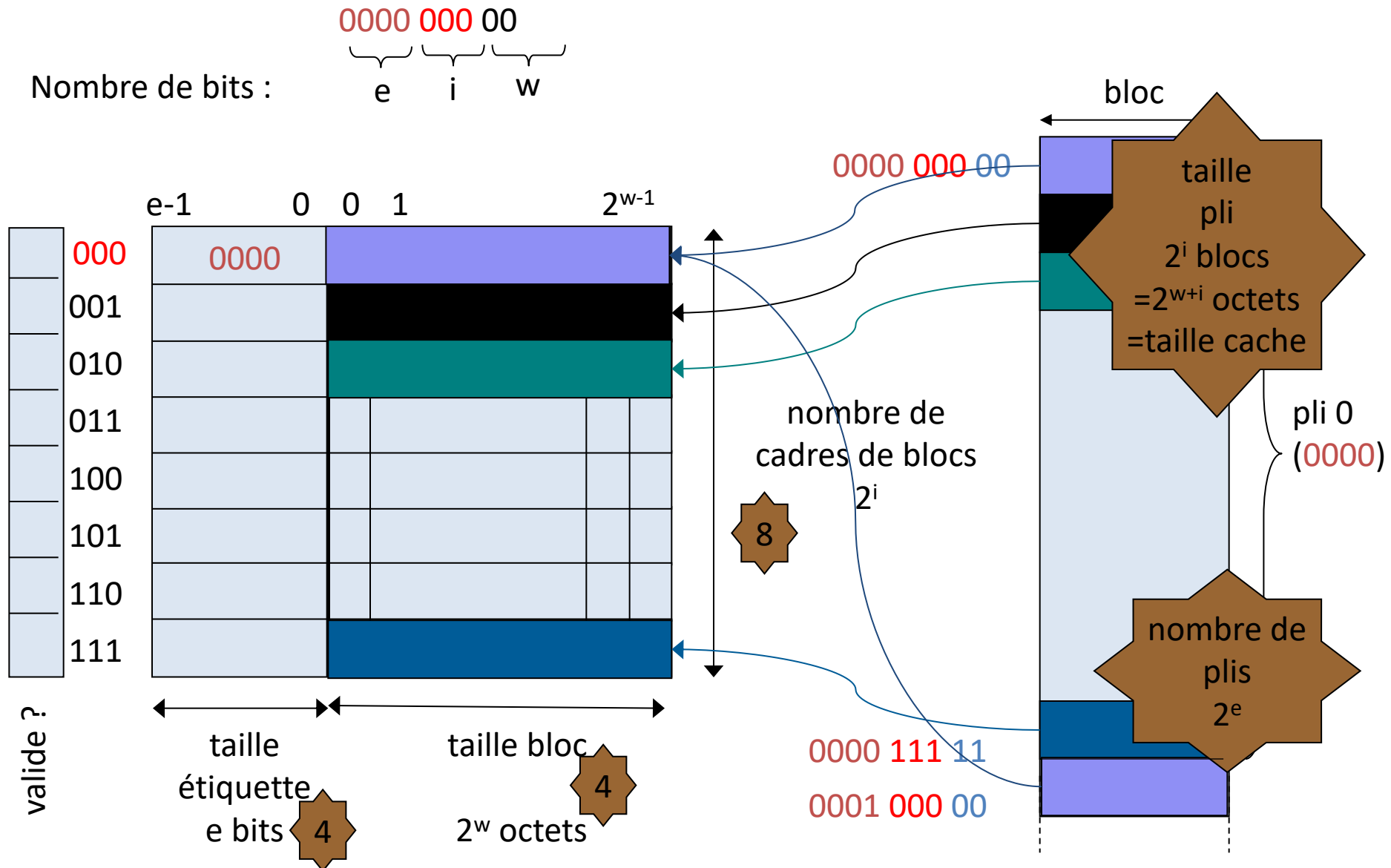
# Algorithme lecture



# Architecture d'un cache à correspondance directe



# Correspondance directe



# Correspondance directe

- Où le bloc doit-il aller dans le cache ?
  - dans le cadre de n° i
- Savoir si un bloc est dans le cache
  - si l'étiquette dans le cadre n° i = étiquette de l'adresse
- Cas d'une écriture
  - succès
    - écriture simultanée (*write-through*) dans le cache et en mémoire
      - tampon d'écriture
    - écriture différée en mémoire (réécriture *write-back*)
      - écriture en mémoire lors du remplacement du bloc par un autre
  - échec
    - écriture allouée
      - si échec, bloc copié dans le cache
    - écriture non allouée
      - écriture seulement en mémoire

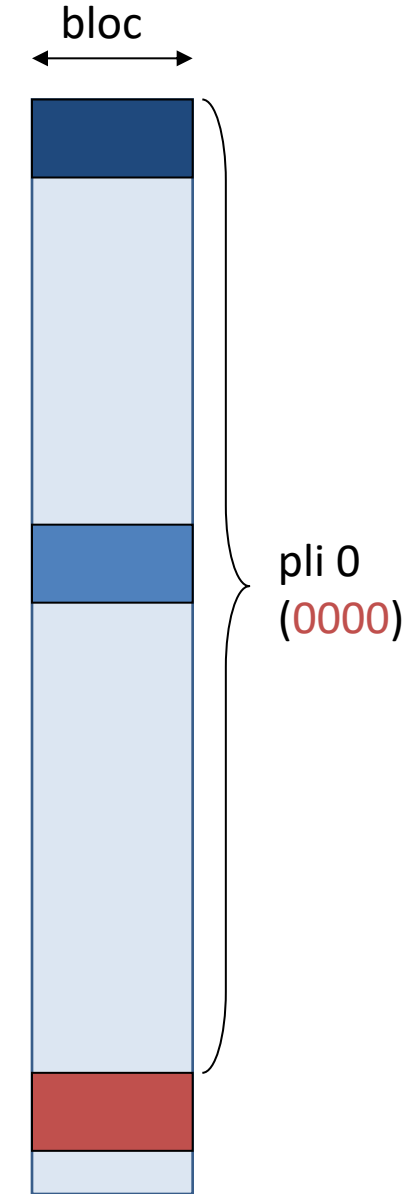
# Associativité par ensembles (de 2)

0000 000 00      0001 000 00  
 e      i      w  
 = n° ensemble

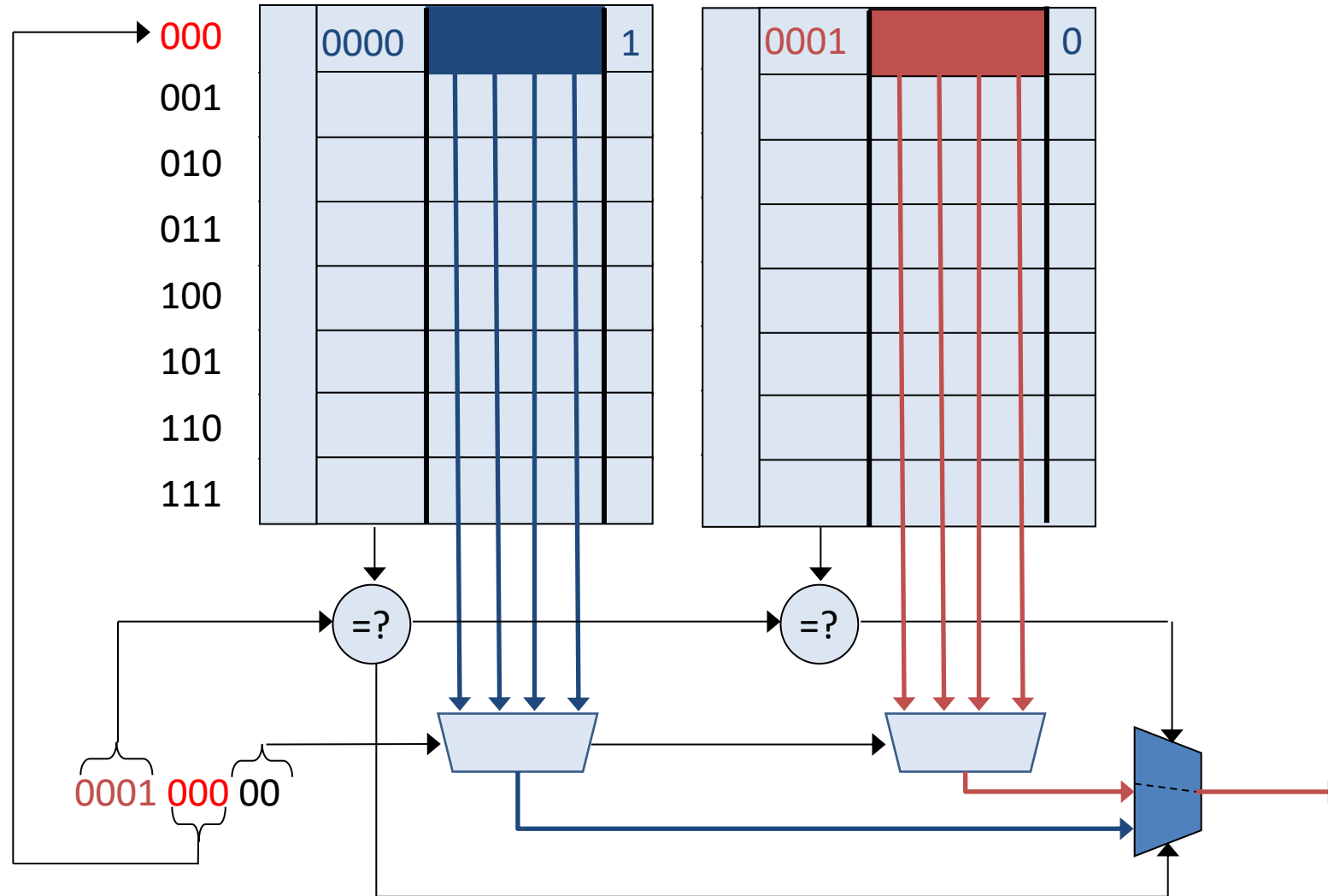
Nombre de bits :

1001 000 00

|     |          |           |         |  |          |           |         |
|-----|----------|-----------|---------|--|----------|-----------|---------|
| 000 |          | 0000      |         |  |          |           | 0       |
| 001 |          |           |         |  |          |           |         |
| 010 |          |           |         |  |          |           |         |
| 011 |          |           |         |  |          |           |         |
| 100 |          |           |         |  |          |           |         |
| 101 |          |           |         |  |          |           |         |
| 110 |          |           |         |  |          |           |         |
| 111 |          |           |         |  |          |           |         |
|     | valide ? | étiquette | contenu |  | valide ? | étiquette | contenu |
|     |          |           |         |  |          |           | LRU     |



# Associativité par ensembles (de 2)



# Associativité par ensemble

- Où le bloc doit-il aller dans le cache ?
  - dans un des cadres de l'ensemble n° i
  - un des 2 au hasard ou LRU
- Savoir si un bloc est dans le cache
  - si l'étiquette d'un des cadres de l'ensemble n° i = étiquette de l'adresse
- Cas d'une écriture *Besoin de + de comparateurs et d'un multiplexeur*
  - idem correspondance directe

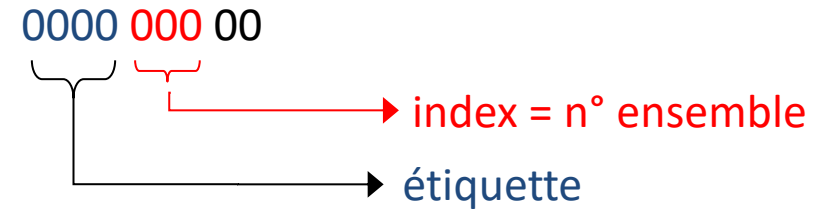
| associativité | 2 cadres/ens. |        | 4 cadres/ens. |        | 8 cadres/ens. |        |
|---------------|---------------|--------|---------------|--------|---------------|--------|
| Taille        | LRU           | hasard | LRU           | hasard | LRU           | hasard |
| 16 Ko         | 5,18%         | 5,69%  | 4,67%         | 5,29%  | 4,39%         | 4,96%  |
| 64 Ko         | 1,88%         | 2,01%  | 1,54%         | 1,66%  | 1,39%         | 1,53%  |
| 256 Ko        | 1,15%         | 1,17%  | 1,13%         | 1,13%  | 1,12%         | 1,12%  |

# Associativité totale

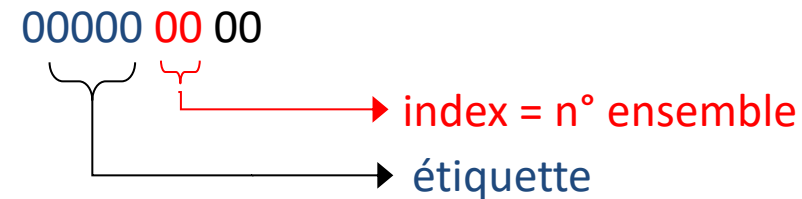
- Cache = un ensemble de  $n$  cadres
- Un bloc peut aller dans n'importe quel cadre
- Un bloc est-il dans le cache ?
  - Comparaison des étiquettes de tous les cadres !

# Continuum d'associativité

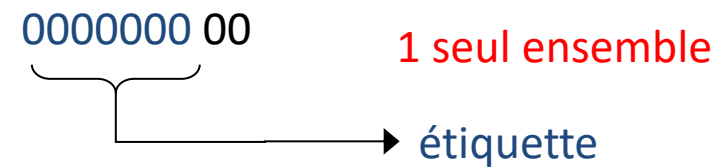
Accès direct  
n ensembles de 1 cadre



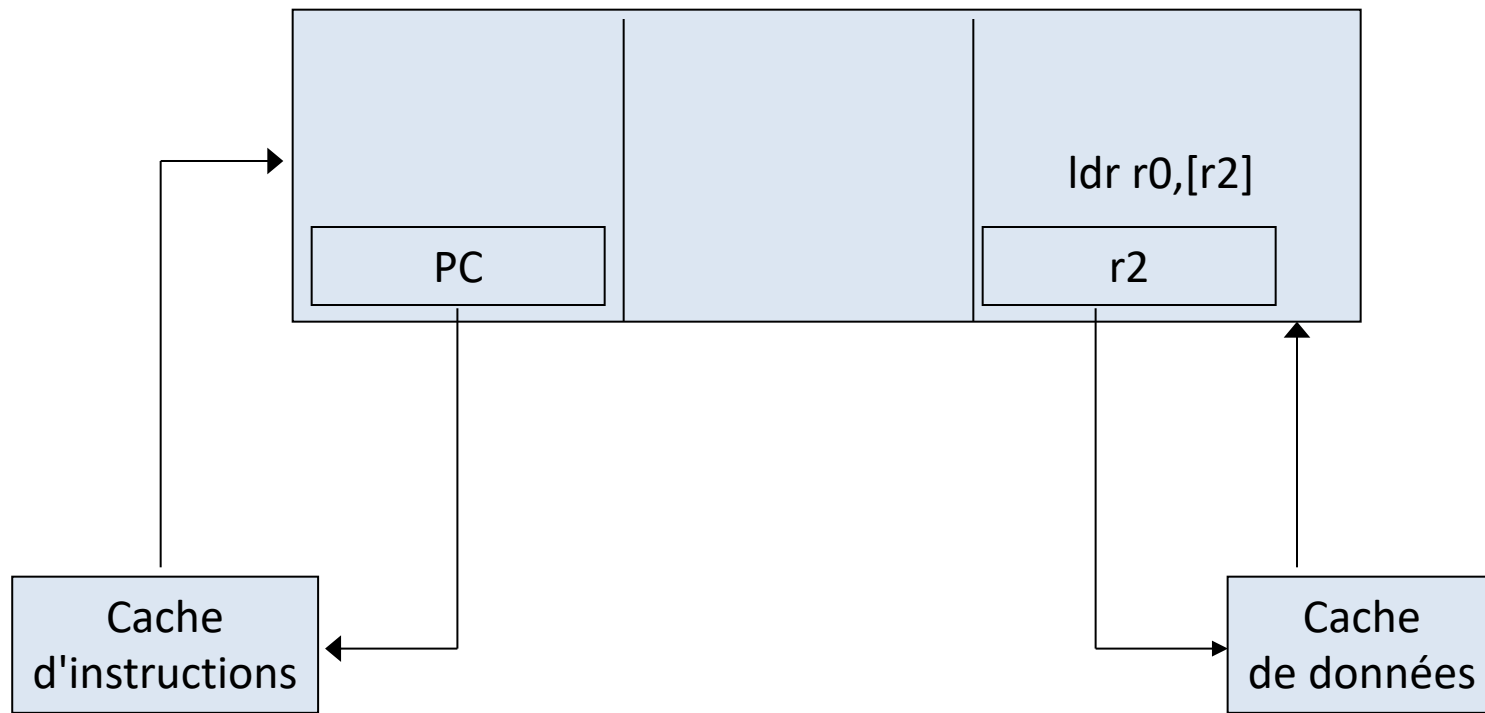
Associativité par ensemble  
n/a ensembles de a cadres



Associativité totale  
1 ensemble de n cadres

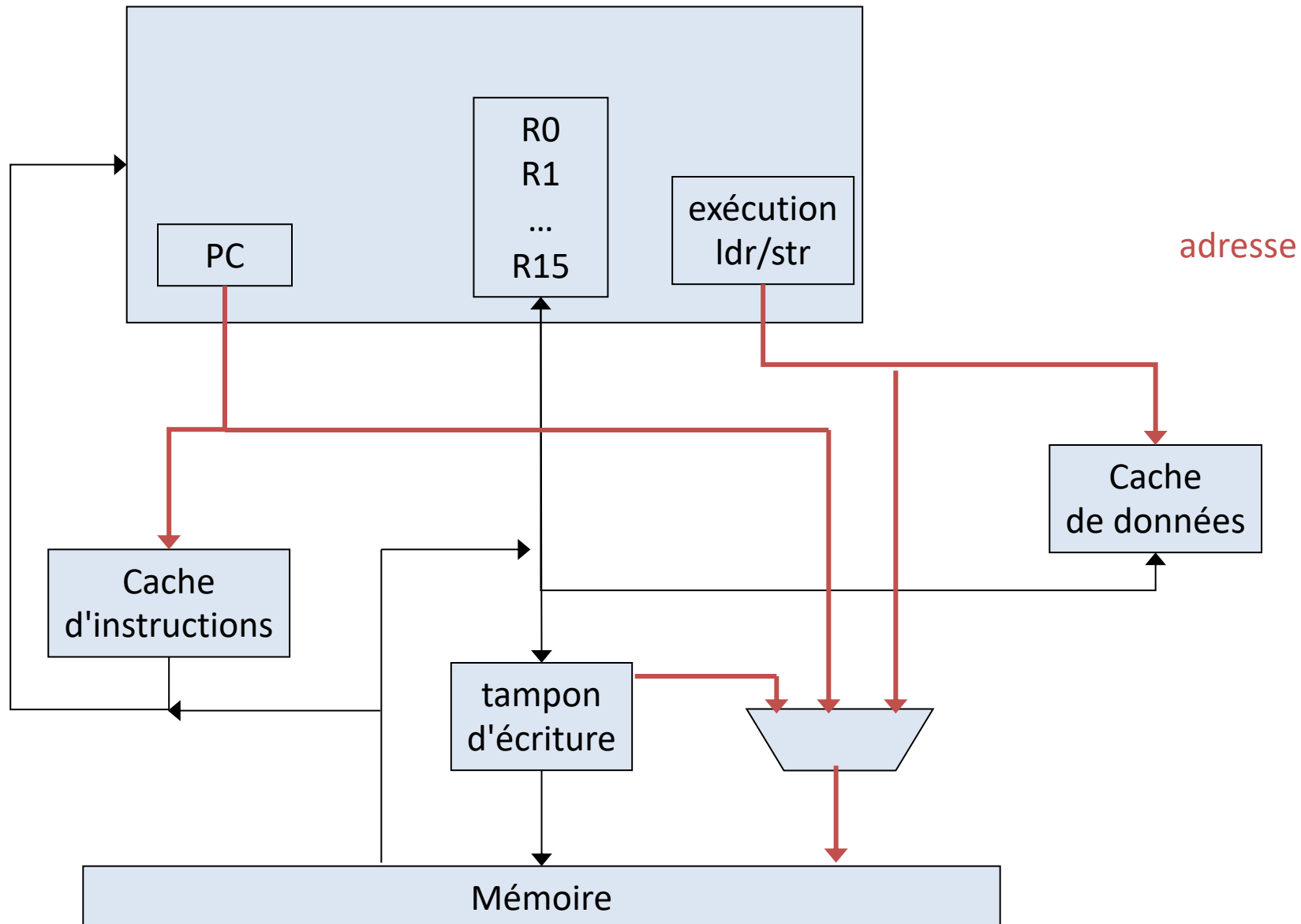


# Cache d'instructions et cache de données



Parallélisme

# Architecture Harvard



# Pourquoi ça marche un cache ?

- Localité temporelle
  - accès à une variable
  - boucles
- Localité spatiale
  - parcours d'un tableau
  - boucles
- Lecture cmp
  - ramène cmp, bcs, str, ...
- Accès tab[0]
  - ramène dans le cache tab[0] à tab[x] ( $x = f(\text{taille bloc})$ )
- Il y a toujours un échec la première fois
  - *Echec obligatoire* ou de première référence ou de démarrage à froid

```
adr    r0,tab
mov    r1,#0
mov    r2,#0
tq:    cmp    r1,#N
        bcs    fin
        str    r2,[r0],#4
        add    r1,r1,#1
        b      tq
fin:    ...

tab:    00 00 00 00
        00 00 00 00
        ...
```

# Pourquoi ça ne marche pas toujours ?

- Cache d'instructions à accès direct
- `cmp` et `tab[0]` tombe dans le même cadre !
  - *Echecs de conflit* permanent

```
adr    r0,tab
mov    r1,#0
mov    r2,#0
tq:    cmp    r1,#N
        bcs    fin
        str    r2,[r0],#4
        add    r1,r1,#1
        b      tq
fin:    ...

tab:    00 00 00 00
        00 00 00 00
        ...
```

# Pourquoi ça ne marche pas toujours ?

- Cache de données à accès direct
  - Addition de deux tableaux
  - Chaque tableau a la même taille que le cache et sont consécutifs en mémoire
- Accès à tab1[0]
  - Charge tab1[0] à tab1[3]
- Accès à tab2[0]
  - Remplace tab1[0..3] par tab2[0..3]
- ...
- Idem si les tableaux ne sont pas alignés sur les plis !  
*Echecs de conflit et de capacité*



# Performance de la mémoire cache

- Taux de succès (%)
- Taux d'échecs =  $1 - \text{taux de succès (\%)}$
- Temps d'accès en cas de succès (cycles d'horloge  $\mu\text{P}$ )
  - en lecture, en écriture
- Temps d'accès en cas d'échec (cycles d'horloge  $\mu\text{P}$ )
  - en lecture, en écriture
- Bande passante
- ...

# Performance de la mémoire cache

- temps  $\mu P$  = nombre d'instructions x (cycles par instruction + (accès mémoire/instruction) \* taux d'échecs \* pénalité d'échecs) \* temps de cycle
- temps  $\mu P$  = nombre d'instructions x (cycles par instruction + (cycles d'attente mémoire / instruction) \* temps de cycle
- cycles d'attente mémoire =  
nombre de lectures \* taux d'échecs lecture \* pénalité échec lecture + nombre d'écritures \* taux d'échecs écriture \* pénalité échec écriture
  - Souvent simplifié en :
- cycles d'attente mémoire =  
nombre d'accès \* taux d'échecs \* pénalité échec

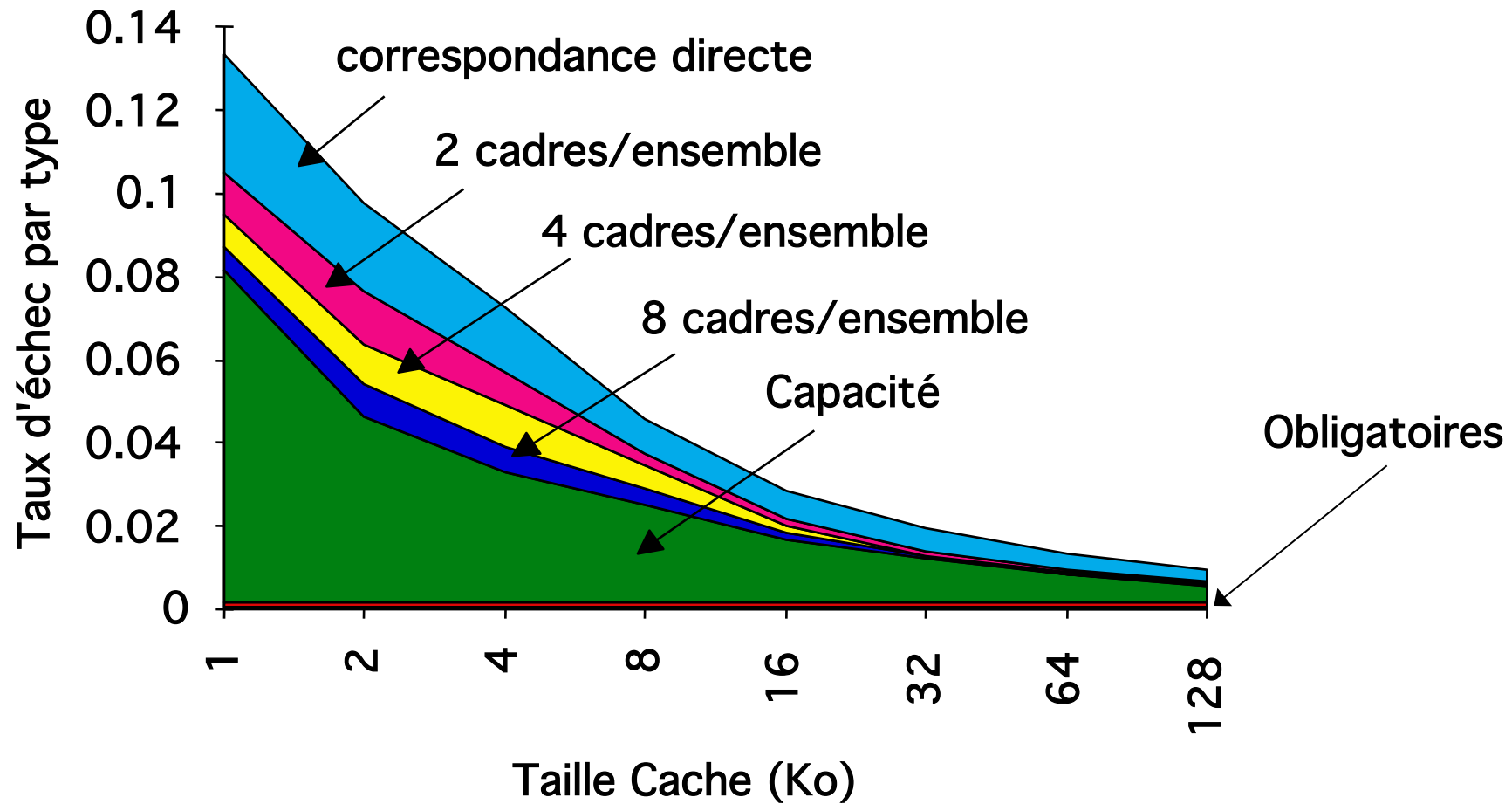
# Exemple

- temps  $\mu P$  = nombre d'instructions \* (cycles par instruction + (cycles d'attente mémoire / instruction)\*temps de cycle
- nombre de cycles par instruction avec cache parfait : 2  
pénalité échec : 50 cycles  
1,33 référence mémoire par instruction  
taux d'échecs : 2%
- temps  $\mu P$  =  $NI * (2 + 1,33 * 2\% * 50) * \text{temps de cycle}$   
=  $NI * 3,33 * \text{temps de cycle}$
- temps  $\mu P$  sans cache =  $NI * 2 + (50 * 1,33) * \text{temps de cycle} = NI * 68,5 * \text{temps de cycle}$

# Performance de la mémoire cache

- temps d'accès mémoire moyen =  
temps accès réussi + taux d'échecs \* pénalité d'échec
- réduire le taux d'échecs
- réduire la pénalité d'échec
- réduire le temps d'accès réussi au cache

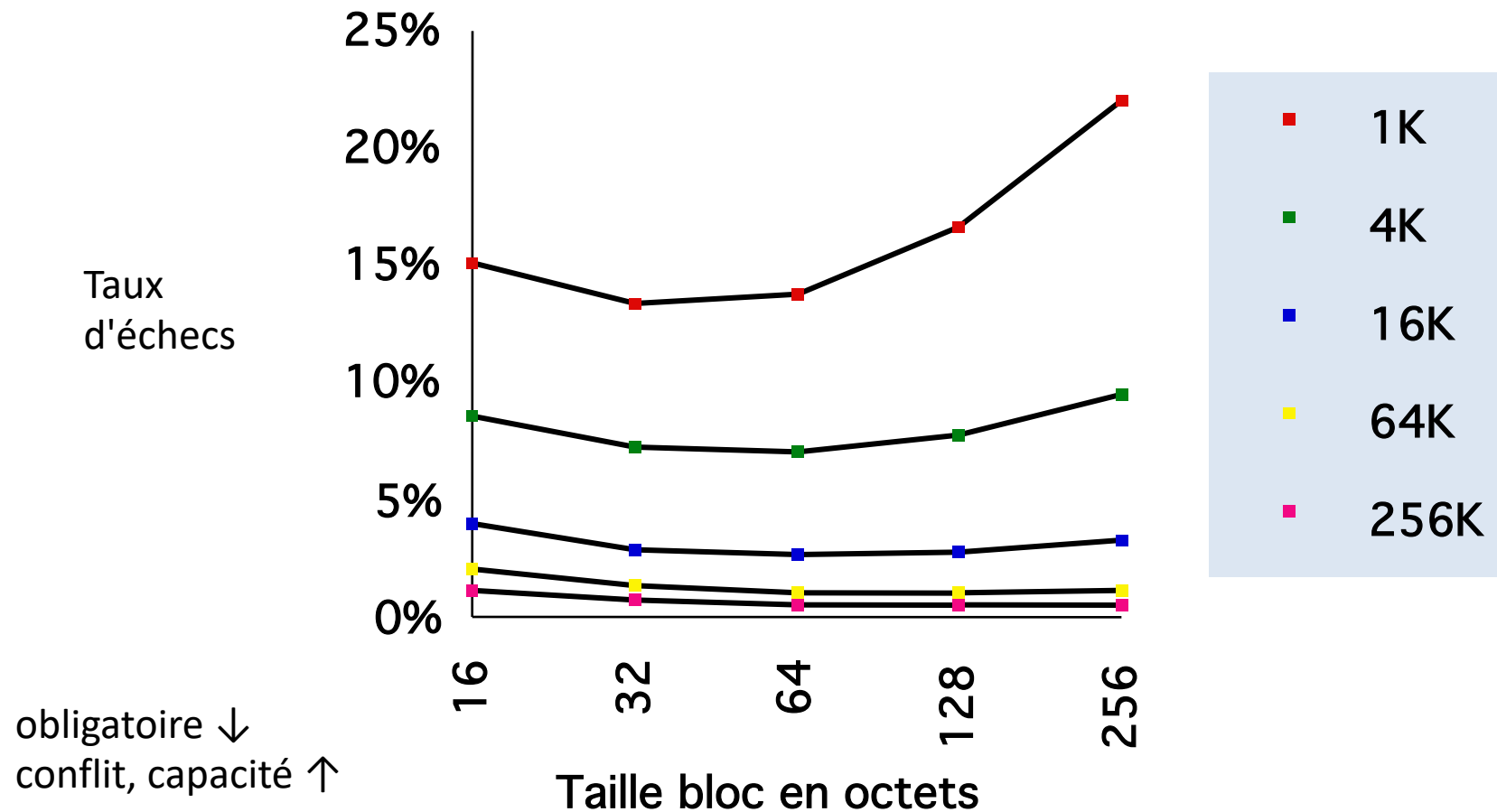
# taux d'échecs par types



# réduire le taux d'échecs

- A taille de cache constante
- Augmenter la taille du bloc
  - obligatoire, capacité, conflit ?
- Changer l'associativité
  - obligatoire, capacité, conflit ?
- Changer l'algorithme (ou le compilateur)
  - obligatoire, capacité, conflit ?

# Augmenter la taille du bloc



intérêt dépend de la latence et de la bande passante (i.e. pénalité)

# Plus grande associativité

- associativité augmentée = temps d'accès augmenté...
  - de direct à associativité de 2 :
    - interne : +2%
    - externe : + 10 %
- Exemple page suivante :
  - assoc. 2 : +10%
  - assoc. 4 : +12%
  - assoc. 8 : +14%

# Plus grande associativité

| Taille Cache |       | Associativité |             |             |             |
|--------------|-------|---------------|-------------|-------------|-------------|
| (Ko)         | 1-way | 2-way         | 4-way       | 8-way       |             |
| 1            |       | 2.33          | 2.15        | 2.07        | 2.01        |
| 2            |       | 1.98          | 1.86        | 1.76        | 1.68        |
| 4            |       | 1.72          | 1.67        | 1.61        | 1.53        |
| 8            |       | 1.46          | <u>1.48</u> | <u>1.47</u> | 1.43        |
| <u>16</u>    |       | <u>1.29</u>   | <u>1.32</u> | <u>1.32</u> | <u>1.32</u> |
| <u>32</u>    |       | <u>1.20</u>   | <u>1.24</u> | <u>1.25</u> | <u>1.27</u> |
| <u>64</u>    |       | <u>1.14</u>   | <u>1.20</u> | <u>1.21</u> | <u>1.23</u> |
| <u>128</u>   |       | <u>1.10</u>   | <u>1.17</u> | <u>1.18</u> | <u>1.20</u> |

Pas d'amélioration du temps d'accès moyen à la mémoire

# Lecture anticipée

- Matérielle
  - chargement automatique du bloc  $i+1$  quand accès au bloc d'instructions  $i$ 
    - Suppose une localité spatiale plus grande que la taille d'un bloc
    - Ne fonctionne pas bien si beaucoup de branchements
    - Nécessite une grande bande passante entre la mémoire et le cache
    - Le bloc  $i+1$  peut être placé dans un tampon consulté lors d'un échec
      - pour ne pas écraser des instructions utiles dans le cache
- Logicielle
  - instructions de préchargement de données
    - initialisation d'un tableau

# Optimisations du code

- Réduction des échecs de conflit
- Instructions
  - Mieux placer les procédures en mémoire
- Données
  - Fusion de tableaux
  - Echange de boucles
  - Fusion de boucles
  - blocage

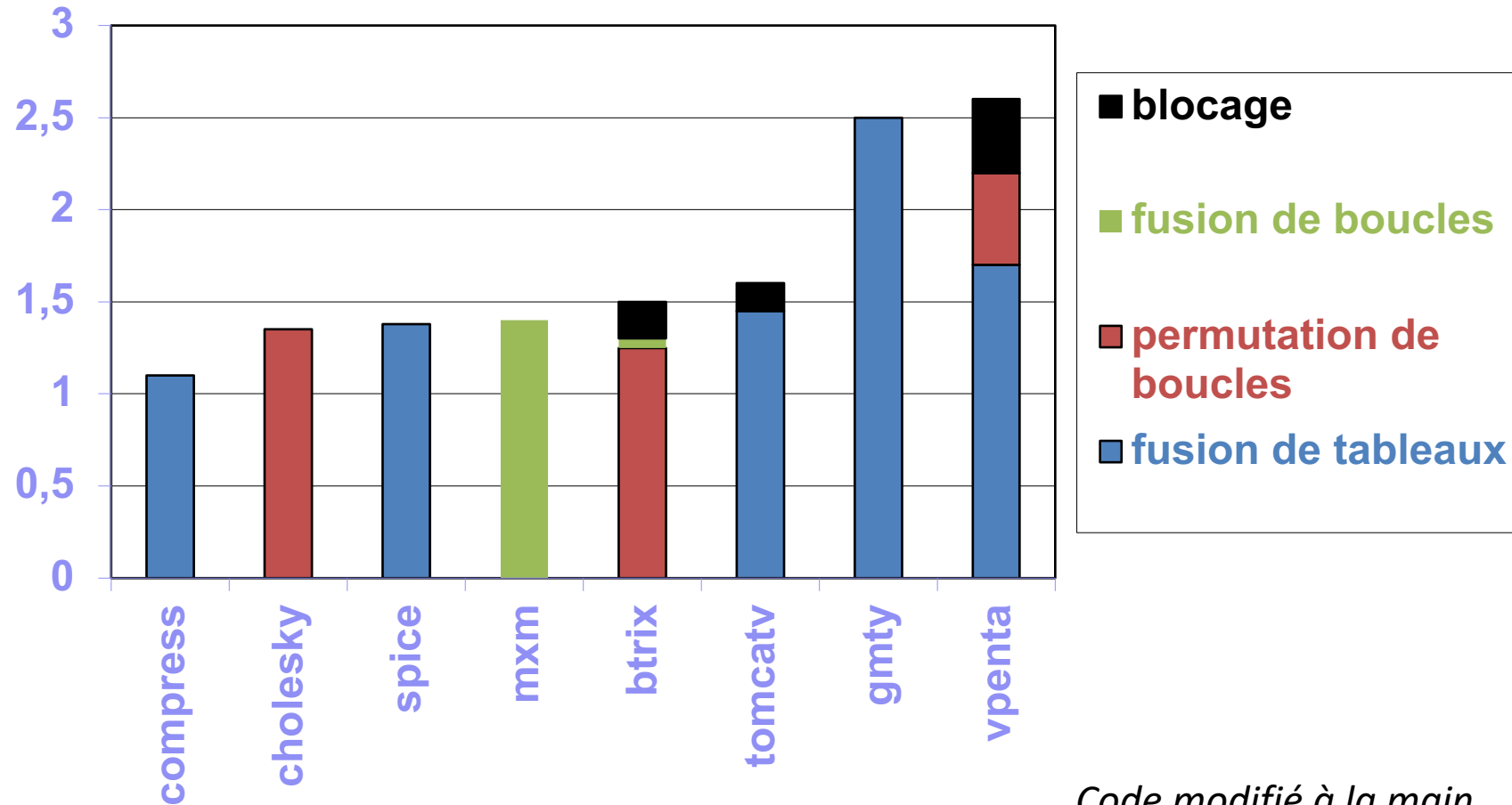
# Exemple de fusion de tableaux

```
int val1[taille], val2[taille];  
for (i=0; i<taille ; i++)  
    val1[i]=val1[i]+val2[i];
```

```
int val[2*taille];  
for (i=0; i<2*taille; i=i+2)  
    val[i] = val[i] + val[i+1];
```

- Augmente la localité spatiale
- Evite des conflits si grands tableaux

# Performance de l'optimisation de code



*Code modifié à la main*

# tampon d'écritures

- Eviter de bloquer le processeur en attendant la fin de l'écriture en mémoire
  - Ecriture simultanée (write-through)
    - écriture dans le cache et dans le tampon
    - en cas d'échec en lecture, vérifier que la donnée n'est pas dans le tampon avant d'accéder à la mémoire
  - Réécriture (write-back)
    - lecture en échec
      - nécessité de réécrire en mémoire un bloc modifié dans le cache
      - mémoriser les blocs modifiés
    - Réécriture dans un tampon, lecture en mémoire (débloque le processeur), écriture du tampon dans la mémoire

# Redémarrage précoce et mot demandé en tête

- échec – copie du bloc mémoire dans le cache – envoi par le cache **du mot demandé** au processeur



- échec – copie du bloc mémoire dans le cache et envoi du mot demandé au processeur dès qu'il arrive



- échec – copie du bloc mémoire dans le cache avec le mot demandé en tête et envoi du mot au processeur dès qu'il arrive



# Cache de niveau 2 (L2)

- L2 Equations

temps d'accès mémoire moyen =  $\text{TempsSuccès}_{L1} + \text{TauxEchecs}_{L1} \times \text{PénalitéEchec}_{L1}$

$\text{PénalitéEchec}_{L1} = \text{TempsSuccès}_{L2} + \text{TauxEchecs}_{L2} \times \text{PénalitéEchec}_{L2}$

temps d'accès mémoire moyen =  $\text{TempsSuccès}_{L1} + \frac{\text{TauxEchecs}_{L1}}{\text{PénalitéEchec}_{L2}} \times (\text{TempsSuccès}_{L2} + \text{TauxEchecs}_{L2} \times \text{PénalitéEchec}_{L2})$

- Définitions:

- *Taux d'échecs local* — échecs dans ce cache divisé par le nombre total d'accès à ce cache (Miss rate<sub>L2</sub>)
- *Taux d'échecs global* — échecs dans ce cache divisé par le nombre total d'accès engendré par le CPU ( $\text{TauxEchecs}_{L1} \times \text{TauxEchecs}_{L2}$ )
- Ci-dessus ce sont les taux d'échecs locaux qui sont utilisés mais ce qui nous intéresse est le taux d'échecs global

# Taux d'échec local et global

- Le processeur génère 1000 accès à la mémoire
- 40 échecs au premier niveau de cache
- 20 échecs au second niveau de cache
- taux d'échecs (local=global) L1 =  $40/1000 = 4\%$
- taux d'échecs local L2 =  $20/40 = 50\%$
- taux d'échecs global L2 =  $20/1000 = 2\%$

# En résumé

| Technique                        | TE | PE | TS | Complexité |
|----------------------------------|----|----|----|------------|
| Taille de bloc plus grande       | +  | -  |    | 0          |
| Plus grande associativité        | +  |    | -  | 1          |
| Préchargement matériel           | +  |    |    | 2          |
| Préchargement logiciel           | +  |    |    | 0          |
| Priorité aux échecs en lecture   |    | +  |    | 1          |
| Mot demandé en tête              |    | +  |    | 2          |
| Cache de 2 <sup>ème</sup> niveau |    | +  |    | 2          |

TE : taux d'échecs, PE : pénalité d'échecs, TS : Temps d'accès succès